Epoka University
Department of Computer Engineering

Course Name: Parallel Programming
Course Code: CEN 330

Project Title:

PageRank Algorithm with Structured Parallelism

Data Parallelism · Task Parallelism · A Live Visualiser

Submitted by:

- **Viola Zeneli**
- **Sidrit Rrustemi**
- **Arsildo Veliu**
- **Megi Almadhi**
- **Artemisa Hasalami**

Lecturer: Shkumbin Fida
Submission Date: 30 / 05 / 2026

• TABLE OF CONTENTS •

Where to find what

PageRank, briefly 3

The formula 3
Why iterate 3

A tiny e-commerce site, 15 pages 4

The site, drawn out 4
In code 5

Data parallelism 6

What each worker does 6
The driver loop 6

Task parallelism 7

The three tasks 7
The two harnesses 7

What the numbers actually say 9

Top pages by PageRank 9
Speedup, side by side 9
Reading the numbers 10

An interactive visualiser, in the browser 11

What it shows 11
Design and tech 11

01 · BACKGROUND

PageRank, briefly

PageRank was published by Larry Page and Sergey Brin in 1998. It's the idea that made Google's first search engine work, and it boils down to a single sentence: ***a page is important if other important pages link to it.*** That circular definition is what makes it interesting — you can't rank one page without knowing the ranks of all the pages that link to it.

The formula

For every page p in a graph of N pages, PageRank computes:

```
PR(p) = (1 - d) / N  +  d × Σ ( PR(q) / outlinks(q) ) , for every q that links to p
```

The first term is a small constant — every page gets a tiny baseline rank just for existing. The second term is the interesting part: every page that points at p gives p a fraction of its own rank, divided among all the places it links to. The damping factor d (we use the classic `0.85`) models the probability that a random surfer follows a link rather than jumping to a random page.

> *In plain language* — **Damping factor**
> If a random web surfer is reading a page, there's an 85% chance they click a link on it and a 15% chance they get bored and jump to a random page somewhere else. That 0.85 is the damping factor. It keeps the algorithm from getting stuck in dead-ends, and it's what makes the maths converge.

Why iterate

You can't compute PageRank in one shot, because every page's rank depends on every other page's rank. So you start with everyone equal — `1/N` each — and apply the formula above. The first pass gives you bad ranks. The second pass uses those bad ranks to compute slightly less bad ranks. After about 20 passes the numbers stop changing meaningfully, and you're done. We use 20 iterations in this project, which is more than enough to converge for a 15-node graph.

02 · THE TEST GRAPH

A tiny e-commerce site, 15 pages

Real web graphs are huge — billions of pages, trillions of links. For a class project we needed something small enough to reason about by hand but rich enough that the rankings would feel non-trivial. We sketched a small e-commerce site: a homepage, a few product pages, a shopping cart, a checkout flow, plus the usual About / Contact / Blog. Fifteen pages and thirty-nine directed links in total. The result is a graph where **Products** and **Cart** end up dominating, because almost everything links to them.

The site, drawn out

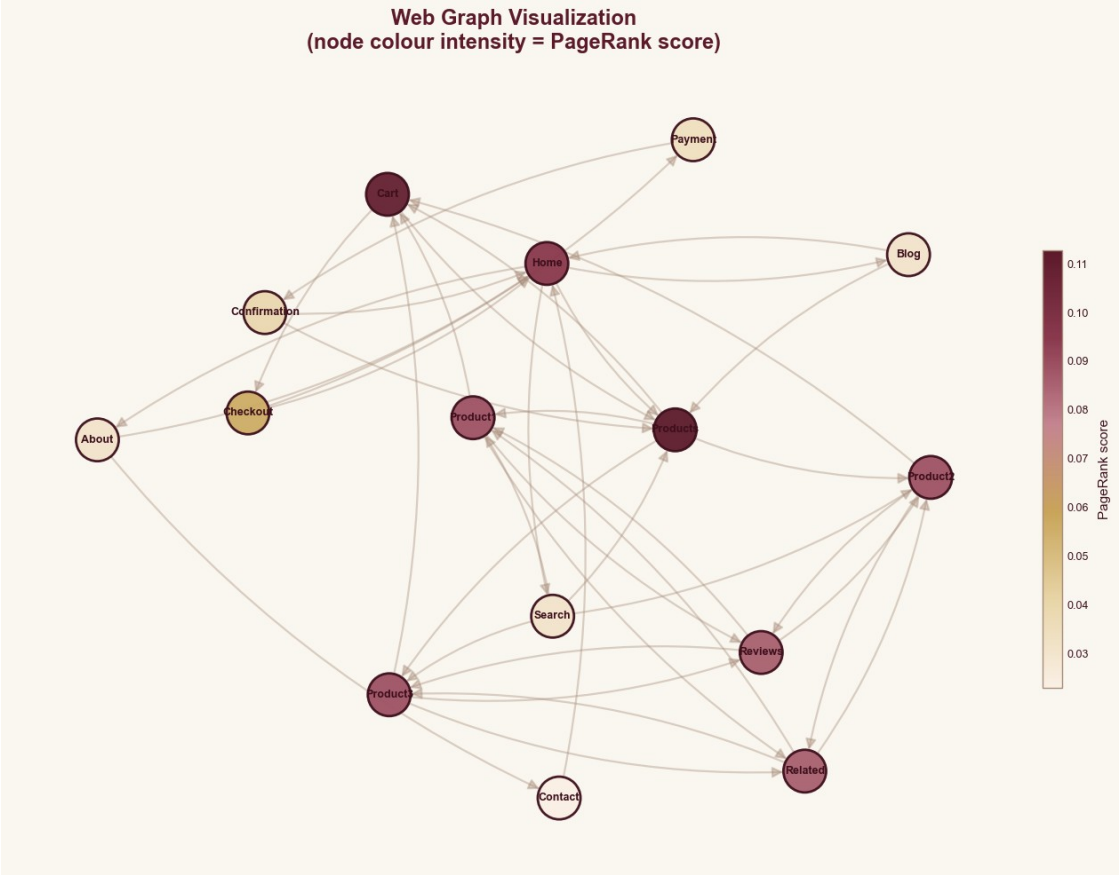


FIG. *The 15-page e-commerce graph. Node colour intensity tracks PageRank — the deep burgundy nodes (Products, Cart, Home) are the most-linked-to pages on the site.*

A few things to notice from the figure. The strongest hubs in terms of outgoing links — **Search, Home, Products** — are the navigational pages that send visitors into the catalogue. The `Product1 / Product2 / Product3` detail pages link tightly to each other through `Reviews` and `Related`, which creates a feedback loop that pumps rank into the product cluster. **Cart** sits at the centre because the entire purchase funnel routes through it.

In code

We represented the site as a Python dictionary mapping each page to the list of pages it links to:

```
web_graph = {
    'Home':       ['Products', 'Search', 'About', 'Blog'],
    'Products':   ['Product1', 'Product2', 'Product3', 'Cart'],
    'Product1':   ['Cart', 'Reviews', 'Related'],
    'Cart':       ['Checkout', 'Products'],
    'Checkout':   ['Payment', 'Home'],
    'Payment':    ['Confirmation'],
    'Confirmation': ['Home', 'Products'],
    # ... 8 more pages
}
```

From that dictionary we build a **`15 × 15`** adjacency matrix where `A[d, s] = 1 / outlinks(s)` whenever *s* links to *d*. For the timing benchmarks we then tile this matrix into a bigger 750-node graph (the small one finishes too fast to time reliably), but the rankings shown in this report all come from the 15-node original.

03 · METHOD 1

Data parallelism

The first parallel pattern we applied is the one that fits PageRank naturally: ***split the data, keep the operation the same.*** Every iteration of PageRank does exactly the same arithmetic on every row of the adjacency matrix. So we hand each thread a slice of rows and let them all run the same code on different data, then stitch the results back together.

> *In plain language* — **Data parallelism**
> Same recipe, four cooks, different ingredients. Each worker runs the identical code, just on a different slice of the input. The opposite of task parallelism, where each worker runs different code.

What each worker does

Each worker receives a list of node indices — its *chunk* — and computes the new rank for just those nodes:

```
def compute_ranks_chunk(node_indices, adjacency, ranks, damping, n):
     chunk_ranks = (1 - damping) / n + damping * (adjacency[node_indices, :] @ ranks)
     return node_indices, chunk_ranks
```

The expression `adjacency[node_indices, :] @ ranks` is a single NumPy matrix-vector multiply on a sub-rectangle of the matrix — fast and vectorised. The driver loops 20 times (one per PageRank iteration), submits all chunks to the thread pool, waits for them to finish, assembles the new rank vector, and repeats.

The driver loop

```
def pagerank_data_parallel(adjacency, n_iter=20, damping=0.85, n_workers=4):
    n = adjacency.shape[0]
    ranks = np.ones(n) / n
    chunks = [c for c in np.array_split(np.arange(n), n_workers) if len(c) > 0]

    # Pool is created ONCE, reused for every iteration
    with ThreadPoolExecutor(max_workers=n_workers) as executor:
    for _ in range(n_iter):
        new_ranks = np.zeros(n)
        futures=[executor.submit(compute_ranks_chunk,chunk,adjacency,ranks,damping,n)
                for chunk in chunks]
        for fut in futures:
                idx, ch = fut.result()
                new_ranks[idx] = ch
        ranks = new_ranks
    return ranks
```

04 · METHOD 2

Task parallelism

Data parallelism splits one job across several workers. Task parallelism does the opposite — ***several different jobs at the same time.*** A real PageRank pipeline doesn't just compute scores; it also analyses the graph's structure and identifies hub pages. The three tasks don't depend on each other, so there's no reason to wait between them.

> *In plain language* — **Task parallelism**
> Different recipes, different cooks, all working at the same time in the same kitchen. The cook making dessert doesn't wait for the cook making the main course. Each thread runs different code, and they finish whenever they finish.

The three tasks

We defined three independent units of work:

- **`task_compute_pagerank`** — Runs the full PageRank algorithm for 200 iterations on the test graph and returns the final score vector. Pure NumPy.
- **`task_analyze_graph`** — Computes in-degree, out-degree, average degree, max in-degree, and graph density. Pure structural analysis.
- **`task_find_hubs`** — Sorts pages by out-degree and returns the top 5 hubs — the pages that send the most traffic outward.

The two harnesses

To measure the benefit we wrote two harnesses with the same shape, differing only in how they invoke the three tasks:

```
def run_tasks_sequential():
    start = time.perf_counter()
    r1 = task_compute_pagerank(adjacency)
    r2 = task_analyze_graph(adjacency, web_graph)
    r3 = task_find_hubs(adjacency, web_graph)
    return time.perf_counter() - start, r1, r2, r3

def run_tasks_parallel():
    start = time.perf_counter()
    with ThreadPoolExecutor(max_workers=3) as executor:
        f1 = executor.submit(task_compute_pagerank, adjacency)
        f2 = executor.submit(task_analyze_graph,    adjacency, web_graph)
        f3 = executor.submit(task_find_hubs,        adjacency, web_graph)
        r1, r2, r3 = f1.result(), f2.result(), f3.result()
    return time.perf_counter() - start, r1, r2, r3
```

The total speedup we get here is roughly bounded by the slowest single task — that's just Amdahl's Law showing up in person. `task_find_hubs` contains a 100 ms sleep simulating a small I/O operation (writing a hub-list to disk in a real pipeline), and the two faster tasks each contain a 30 ms sleep. Sequentially that's about 160 ms of sleep plus the actual compute time. In parallel, the three sleeps overlap, and the slowest one — 100 ms — sets the floor.

05 · RESULTS

What the numbers actually say

Every number below comes from running the notebook on a 4-core MacBook with NumPy capped to one BLAS thread. Times are medians of 5 runs of the benchmark graph (a 750-node tiling of our 15-node site), so they're stable enough to compare against each other.

Top pages by PageRank

Before talking about timing, here's what PageRank actually thinks about the site. The scores are the same regardless of which version computed them — we double-checked with `np.allclose`, and the sequential and parallel outputs agree to floating-point precision.

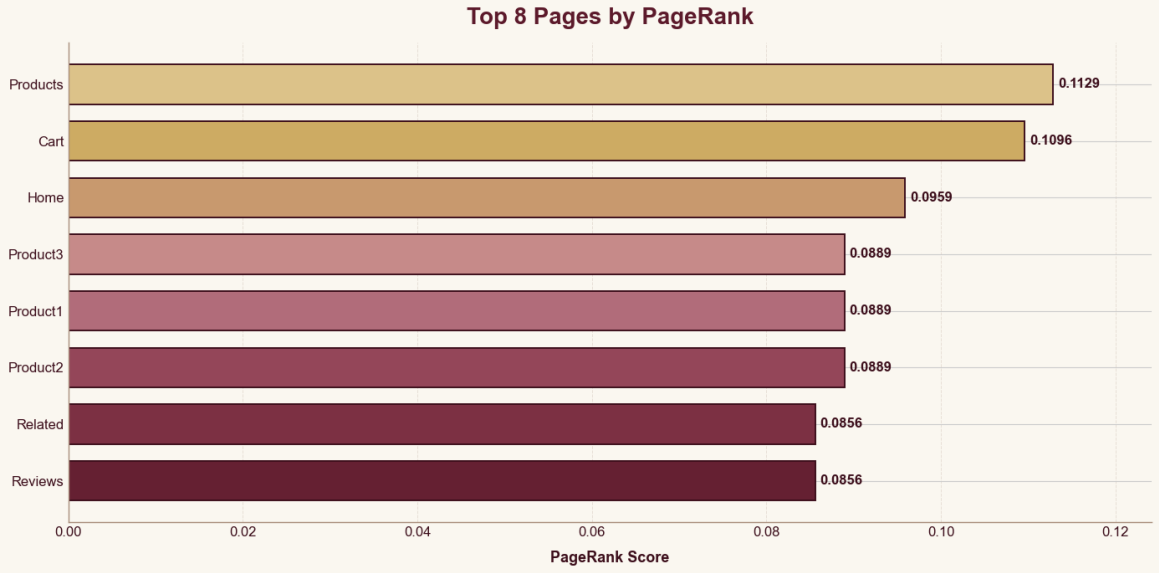


FIG. *Top 8 pages by PageRank score. Products and Cart dominate — almost every other page links to them.*

Products (**`0.1129`**) edges out Cart (**`0.1096`**) and Home (**`0.0959`**), which matches intuition: those three are the most-linked-to pages on the site. The three product detail pages tie at **`0.0889`**, because the graph is symmetric across them.

Speedup, side by side

Same test, two patterns. The left table compares the sequential baseline to the data-parallel runs with 2, 4 and 8 workers. The right table compares the same baseline to task parallelism — the three tasks of Section 05 running concurrently in their own thread pool.

Performance Summary · Speedup by Method

Sequential · Data Parallelism

Configuration	Time	Speedup
Sequential (1 worker)	0.1306 s	**1.00×**
Data parallel · 2 workers	**0.0306 s**	**4.26×**
Data parallel · 4 workers	0.0439 s	**2.97×**
Data parallel · 8 workers	0.0640 s	**2.04×**

Best: Data parallel · 2 workers → 4.26× faster than sequential

Sequential · Task Parallelism

Configuration	Time	Speedup
Sequential (1 task at a time)	0.1921 s	**1.00×**
Parallel (3 tasks concurrent)	**0.1214 s**	**1.58×**

3 concurrent tasks → 1.58× faster than sequential

FIG. *Speedup by method. The gold row in each table marks the best-performing configuration.*

Reading the numbers

Three things stand out.

- **Two workers won.** On a 4-core machine, 2 workers gave us the best data-parallelism speedup at 4.26×. That's better than 4 workers (2.97×), which surprised us at first — but with only 15 nodes split into 4 chunks of ~4 nodes each, the per-chunk work becomes so tiny that the overhead of submitting and collecting futures eats most of the gain. The lighter setup wins.
- **Eight workers oversubscribed.** At 8 workers on 4 physical cores, we're back in oversubscription territory — just at the Python level this time rather than at the BLAS level. The speedup dropped to 2.04×, half what we got with 2 workers. The lesson: matching worker count to physical cores matters more than maxing it out.
- **Task parallelism gave us 1.58×.** Three concurrent tasks dropped wall-clock from 0.19 s to 0.12 s. The ceiling here is whatever the slowest single task takes (about 0.10 s of compute + sleep), and we got reasonably close to that ceiling. Amdahl's Law in action — a parallel program is only as fast as its slowest unparallelisable piece.

06 · BONUS

An interactive visualiser, in the browser

Numbers in a table are convincing, but they're not the easiest thing to feel. We wanted a way to ***see*** the speedup happen rather than read about it, so we built a self-contained HTML file that races a single-worker run against a four-worker run on the same graph, in real time.

What it shows

On the left lane, one amber-coloured worker walks the 15 nodes one at a time. On the right lane, four gold/rose/champagne/mauve workers each handle their own chunk of the graph simultaneously. Both timers tick up live; the iteration counter and PageRank scores update from the real algorithm; only the speed of the animation is scaled so a human can watch it.

A live bar chart at the bottom shows the top five PageRank scores as they converge across the 20 iterations, and the moment both lanes finish, a hero card displays the final ***Sequential / Parallel*** speedup ratio with a generated verdict line. Three small explainer cards below the visualiser remind the reader, in one sentence each, what sequential is, what data parallelism is, and the two-line fix that made any of this possible.

Design and tech

Built with vanilla JavaScript and SVG — no frameworks, no external libraries, single file, no internet needed to view it. The palette intentionally matches this report (deep wine background, royal gold and dusty rose accents, cream typography). The whole layout reflows for tablets and phones thanks to three CSS breakpoints: on a phone the two lanes stack vertically, the four-worker chips collapse into a single column, and the speedup card shrinks but keeps the gradient typography readable.

> ***In plain language*** **— To run it**
> Open pagerank_visualizer.html in any modern browser. There's nothing to install. Hit ‘Play’ to start the race, drag the speed slider to step through more slowly, or click Step iter to advance one iteration at a time.

Two patterns. One graph. Two-line fix.

Thanks for reading!